School of Future Tech

Case Study Report

on

Secure Password Manager

by

Mohammed Saad Mulla

150096725112

Index

1. Introduction to the Case Study.

2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

This case study is about a Secure Password Manager - a command-line application built in C++ using Object Oriented Programming. The system allows users to store, view, update and delete their website credentials in encrypted form. The idea is simple: you log in with a master password, and then safely manage all your saved passwords. No password is ever stored in plain text.

The case study demonstrates how core programming concepts - OOP, file handling, encryption and data management - can be applied together to build a practical and functional security tool using only first year C++ knowledge.

2. Problem Statement / Case Background (Abstract)

Background

With the growing number of online services, users today must manage passwords for dozens of apps and websites. Most people either reuse the same password everywhere or write them down in plain text - both of which are serious security risks that can lead to data breaches, identity theft and unauthorized access. There was a need for a simple, beginner-level tool that stores credentials securely using encryption.

Abstract

This case study presents the design and implementation of a Secure Password Manager using C++ and Object Oriented Programming. The system has a master login with username and password protection, and locks itself after 3 consecutive failed attempts. Passwords are encrypted using a Caesar Cipher (shift of +3) before being written to a file - so they are never stored as plain text. Users can add, view, update and delete credentials through a simple

menudriven interface. Every action is logged with a timestamp, and a report feature shows the total number of stored accounts along with recent activity and last login time.

3. Problem Statement / Case Study Design

The Secure Password Manager is designed as a modular, menu-driven console application. The flow is as follows:

Login Screen - User enters master username and password. Wrong attempts are counted and the system locks after 3 failures.

Main Menu - After successful login, a numbered menu appears with options to add, view, update, delete or report.

Add Credential - User enters website, username and password. Password is encrypted before saving to passwords.txt.

View Credentials - All saved entries are shown with decrypted passwords. StorageManager reads from file and EncryptionManager decrypts.

Update / Delete - User picks an entry number. Password is updated or entry is removed. File is saved and action is logged.

Report - Shows total number of accounts, last login time and last 5 activity log entries.

The project creates 3 files at runtime:

- passwords.txt - stores all encrypted credentials
- log.txt - stores all activity logs with timestamps
- lastlogin.txt - stores the last successful login record

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

OOP Concepts Used

Classes and Objects - The project has 5 classes: EncryptionManager, Credential, User, StorageManager and PasswordManagerSystem. In main(), a PasswordManagerSystem object pm is created and pm.run() is called.

Encapsulation - All data and functions are wrapped inside their respective classes. No data is accessed directly from outside.

Constructors - The Credential class uses a parameterized constructor. The PasswordManagerSystem constructor automatically loads saved credentials from file on startup.
if/else - Used to check login credentials, validate menu choices, and handle wrong attempt count.
for loops - Used to loop through the credentials vector for viewing, updating and deleting entries.

File Handling - ofstream writes credentials and logs to files. ifstream reads them back. ios::app mode appends logs without overwriting.

Vectors - vector stores all credentials in memory. Supports dynamic add and erase operations.

Encryption Algorithm

Caesar Cipher with a shift of +3 is used. Every character in the password is shifted forward by 3 in ASCII value when saving, and shifted back by 3 when reading back. This ensures passwords are never stored as plain text.

Example:

- Original → hello
- Encrypted → khoor

Technology Stack Used for the Case

Technology	Purpose
C++	Core programming language
OOP (Classes)	Modular and clean code structure
fstream (File Handling)	Read and write credentials and logs
vector	Store multiple credentials dynamically in memory
Caesar Cipher	Encrypt and decrypt passwords
ctime library	Add timestamps to every log entry

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation is organized in a single C++ source file with 5 classes, each handling one specific responsibility:

1. EncryptionManager

- encrypt(text) - shifts every character by +3
- decrypt(text) - shifts every character by - 3

2. Credential

- Stores website, username and encrypted password
- Parameterized constructor to initialize all fields

1. User

- Stores master username (Saad) and password (1234)

4. StorageManager

- save() - writes all credentials to passwords.txt
- $\mathrm{load}()$ - reads credentials from file into a vector on startup
- $\log()$ - appends a timestamped entry to log.txt
- saveLastLogin() / getLastLogin() - tracks last login time
- showLogs() - displays last 5 log entries

5. PasswordManagerSystem

- login() - handles authentication and lockout after 3 attempts
- addPassword(), viewPasswords(), updatePassword(), deletePassword() — CRUD operations
- showReport() - shows account count, last login and recent logs
- run() - main loop with menu-driven interface

Snapshots

- Login screen showing username and password prompt

```
mohammedsaadmulla@Saads-M4 C++ proj % cd "/Users/mohammedsaadmulla/C++ proj/" && g++ passwordmanager
.cpp -o passwordmanager && "/Users/mohammedsaadmulla/C++ proj/"passwordmanager
==============================
    Secure Password Manager
==============================
(username: Saad | password: 1234)

Username : Saad
Password : 1234
```

- Main menu with all 6 options listed

```
mohammedsaadmulla@Saads-M4 C++ proj % cd "/Users/mohammedsaadmulla/C++ proj/" && g++ passwordmanager
.cpp -o passwordmanager && "/Users/mohammedsaadmulla/C++ proj/"passwordmanager
==============================
    Secure Password Manager
==============================
(username: Saad | password: 1234)

Username : Saad
Password : 1234

Welcome, Saad!

--- Menu ---
1. Add password
2. View passwords
3. Update password
4. Delete password
5. Report
6. Exit
Choice:
```

- Adding a new credential and confirmation message

```
--- Menu ---
1. Add password
2. View passwords
3. Update password
4. Delete password
5. Report
6. Exit
Choice: 1

Website  : saad.com
Username : saadmulla1
Password : saad@123
Saved!
```

- Viewing stored passwords (decrypted on screen)

```
--- Menu ---
1. Add password
2. View passwords
3. Update password
4. Delete password
5. Report
6. Exit
Choice: 2

--- Your Passwords ---
1. saad.com
   Username : saadmulla1
   Password : saad@123
```

- Report screen showing total accounts and last login

```
--- Menu ---
1. Add password
2. View passwords
3. Update password
4. Delete password
5. Report
6. Exit
Choice: 5

--- Report ---
Total accounts : 1
Last login     : Saad at Sat Mar 14 18:02:35 2026

Recent activity:
  [Sat Mar 14 14:42:51 2026] Login successful - Saad
  [Sat Mar 14 18:02:35 2026] Login successful - Saad
  [Sat Mar 14 18:03:21 2026] Added: saad.com
```

6. Problem Statement / Case Study Results and Conclusion

Findings

After testing the complete application the following was observed:

- Login lockout works correctly - system locks after exactly 3 wrong attempts.
- Passwords saved in encrypted form - passwords.txt shows unreadable cipher text.
- Decryption correctly restores the original password when viewing.
- Log file captures every action with accurate timestamps.
- All CRUD operations (add, view, update, delete) work without data loss.
- Data persists correctly across sessions - credentials from one run load in the next.

Conclusion

This project builds a working Secure Password Manager that demonstrates real-world cybersecurity concepts using only first year C++ knowledge. It uses no external libraries - just standard C++ with classes, vectors and file handling.

The project covers all required OOP concepts, implements a real encryption algorithm and simulates a complete authentication and credential management system. This shows that even basic C++ is enough to build something with genuine practical and educational value.

7. References

- cplusplus.com - C++ standard library reference (fstream, vector, string, ctime)
- GeeksForGeeks - OOP concepts in C++, file handling and class examples
- W3Schools - C++ reference for loops, functions, classes and constructors
- Wikipedia - Caesar Cipher encryption algorithm
- Anthropic Claude - assisted with code structure and documentation writing